

CIS 5357

Computing for Data Analytics

Week 2

25 January 2024

Last week

- Intro to Python
- Welcome quiz

Quiz insights

- Average grade: **99%**
- **Very few issues** installing Python!
- **15% have never done any programming**—don't worry if you don't know another programming language
- **40% have some level of Python experience**—you can be a resource to classmates & help in the discussion board (but no cheating!)
- **Other languages:** SQL, R, Java, C, C++, C#, CSS, HTML, JavaScript, Unix, Visual Basic, PHP, .net, PowerQuery—remember your experience learning these other languages

Terminal command review

- How do you change the folder/directory you are in? `cd`
- What command takes you to the home directory? `cd~`
- What command takes you up one level/folder? `cd..`
- How can you determine which directory you are in? `pwd`
- How can you view the files in the current directory? `ls`
- What keyword do you use to run .py files? `python`

This week

Building blocks of a Python script

- comments
- variables
- data types
- printing

This week

Building blocks of a Python script

- comments
- variables
- data types
- printing

Take input from a user

Debug common errors

Download lecture code in Canvas

Module 2

Read and View

Read

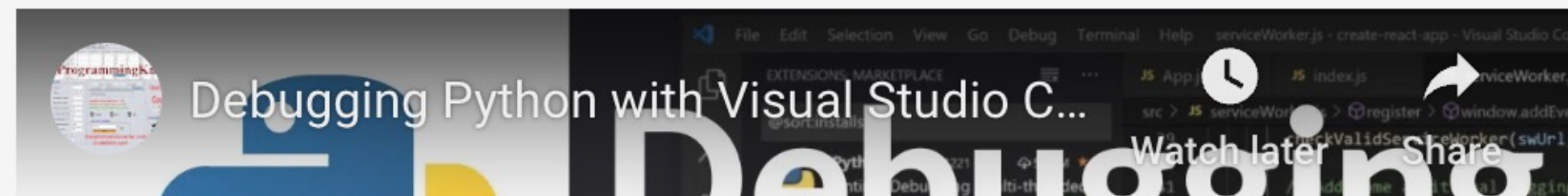
- Textbook chapter 2 (pages 39-70)
- [What Are the Different Types of Python Errors? – and How to Handle Them](#) ➞
- [Python tutorial](#) ➞ sections 3.1.1. Numbers and 3.1.2. Text only
- [Number formats](#) ➞

For reference:

- [Python in Visual Studio Code](#) ➞
- [Practice code with lecture](#) ↓ — follow along in class

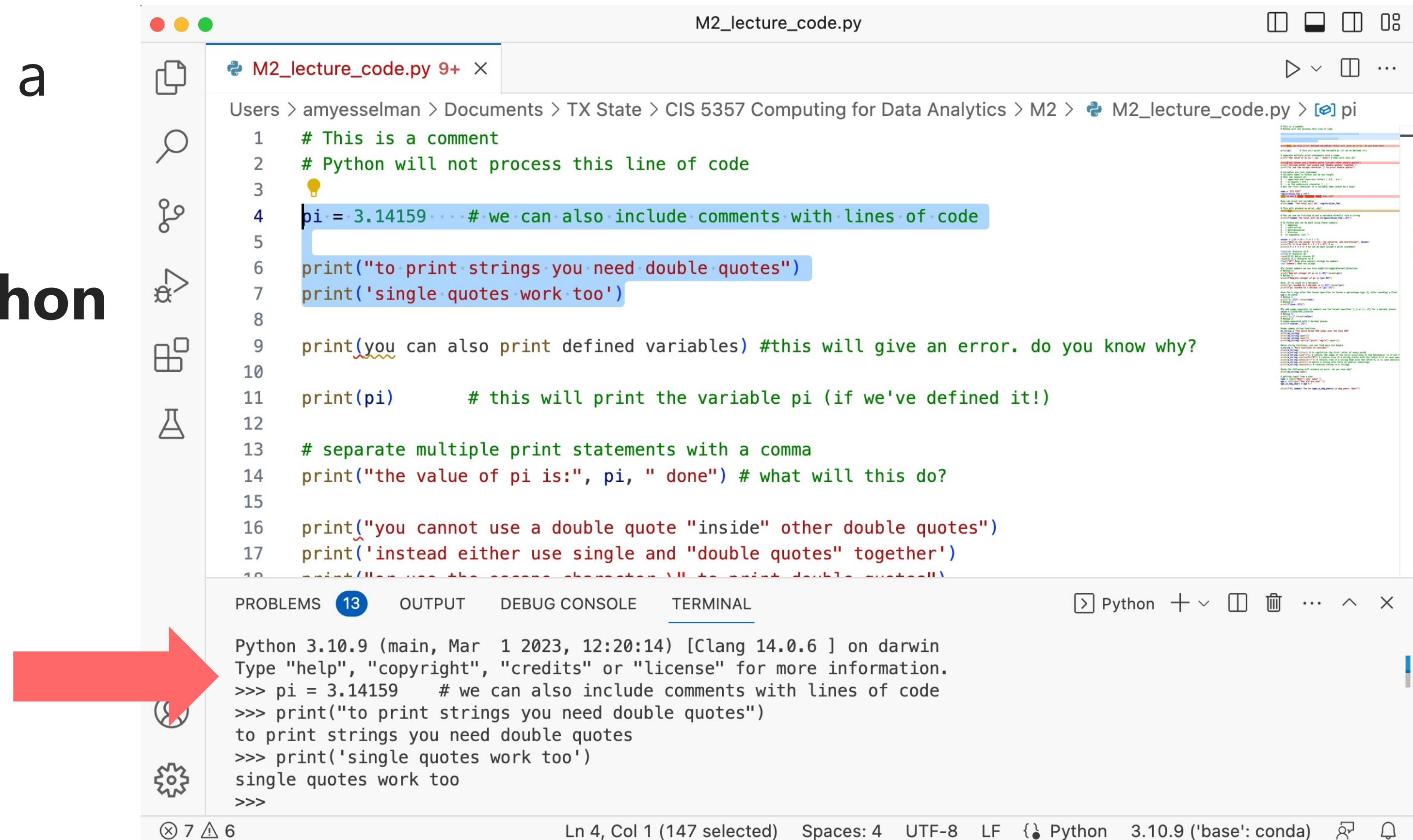
View

[How to debug syntax errors using comments](#) ➞



Run code in Visual Studio Code

Run individual lines or a selection of code with the **Python: Run Selection/Line in Python Terminal** command (**Shift+Enter**)



The screenshot shows the Visual Studio Code interface with a file named `M2_lecture_code.py` open. The file contains the following Python code:

```
1 # This is a comment
2 # Python will not process this line of code
3
4 pi = 3.14159 # we can also include comments with lines of code
5
6 print("to print strings you need double quotes")
7 print('single quotes work too')
8
9 print(you can also print defined variables) #this will give an error. do you know why?
10
11 print(pi) # this will print the variable pi (if we've defined it!)
12
13 # separate multiple print statements with a comma
14 print("the value of pi is:", pi, " done") # what will this do?
15
16 print("you cannot use a double quote "inside" other double quotes")
17 print('instead either use single and "double quotes" together')
18 print("Use the escape character \" to print double quotes")
```

The terminal panel at the bottom shows the output of the code execution:

```
Python 3.10.9 (main, Mar 1 2023, 12:20:14) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> pi = 3.14159 # we can also include comments with lines of code
>>> print("to print strings you need double quotes")
to print strings you need double quotes
>>> print('single quotes work too')
single quotes work too
>>>
```

A red arrow points from the text "Shift+Enter" to the Run button (a play icon) in the terminal panel.

Comments

Lines with a pound sign or hashtag or number symbol (#)

```
1  # This is a comment
2  # Python will not process this line of code
3
4  pi = 3.14159      # we can also include comments with lines of code
5
```

Comments are skipped by Python but are useful for **writing notes** and **debugging**. Please **include comments in your assignments**, including **your name & any other details** that would help me as I review your code.

Printing output

`print()` is a predefined function that returns output on the screen

```
6  print("to print strings you need double quotes")
7  print('single quotes work too')
8
9  print(you can also print defined variables) #this will give an error. do you know why?
10
```

In Python, `\n` is an escape character that will create a new line **when used**. Print on multiple lines without having to use the `print()` function multiple times by using `\n`.

Printing output

print() is a predefined function that returns output on the screen

```
6  print("to print strings you need double quotes")
7  print('single quotes work too')
8
9  print(you can also print defined variables) #this will give an error. do you know why?
10
11 print(pi)          # this will print the variable pi (if we've defined it!)
12
13 # separate multiple print statements with a comma
14 print("the value of pi is:", pi, " done") # what will this do?
15
```

Printing output

`print()` is a predefined function that returns output on the screen

```
6  print("to print strings you need double quotes")
7  print('single quotes work too')
8
9  print(you can also print defined variables) #this will give an error. do you know why?
10
11 print(pi)          # this will print the variable pi (if we've defined it!)
12
13 # separate multiple print statements with a comma
14 print("the value of pi is:", pi, " done") # what will this do?
15
16 print("you cannot use a double quote "inside" other double quotes")
17 print('instead either use single and "double quotes" together')
18 print("or use the escape character \" to print double quotes")
19
```

Also helpful for
debugging!

Variables

```
20  # variables are just nicknames
21  # variable names in Python can be any length
22  # they can consist of:
23  #   - uppercase and lowercase letters ( A-Z , a-z )
24  #   - or digits ( 0-9 )
25  #   - or the underscore character ( _ )
26  # but the first character of a variable name cannot be a digit
27
```

Variables

```
20 # variables are just nicknames
21 # variable names in Python can be any length
22 # they can consist of:
23 #   - uppercase and lowercase letters ( A-Z , a-z )
24 #   - or digits ( 0-9 )
25 #   - or the underscore character ( _ )
26 # but the first character of a variable name cannot be a digit
27
28 name = "CIS 5357"
29 registration_fee = 209.5
30 this is not a valid variable name! #why not?
31
32 #you can print out variables
33 print(name, "the total will be", registration_fee)
34
35 # This will produce an error. why?
36 print(fee)
37
38 # You can use an f-string to put a variable directly into a string
39 print(f"{name} the total will be ${registration_fee:,.2f}")
40
```

Math

```
41  # In Python you can do math using these symbols:
42  #  + addition
43  #  - subtraction
44  #  * multiplication
45  #  / division
46  #  ** exponents (not ^)
47
```

Math

```
41  # In Python you can do math using these symbols:
42  #  + addition
43  #  - subtraction
44  #  * multiplication
45  #  / division
46  #  ** exponents (not ^)
47
48  answer = ((40 + 30 - 7) * 2 / 3)
49  print("What is the answer to life, the universe, and everything?", answer)
50  print("Is it true that 5 * 2 > 3 * 4?") # no
51  print(5 * 2 > 3 * 4) # we can do math inside a print statement
```

Data types

Python works with **values** which come with different **types**

- 42 is an **integer**
- 3.14159 is a **float** (decimal number)
- “bird” is a **string** (a set of characters)

Data types

Python works with **values** which come with different **types**

- 42 is an **integer**
- 3.14159 is a **float** (decimal number)
- “bird” is a **string** (a set of characters)

A variable's type determine what operations can and can't perform

- $2 * 3$ makes sense but what is “cat” * ”dog”?

Type conversions

Sometimes you will need to convert variable types

```
53     float(42) #returns 42.0  
54     int(42.0) #returns 42
```

Type conversions

Sometimes you will need to convert variable types

```
53 float(42) #returns 42.0
54 int(42.0) #returns 42
55 round(42.0) #also returns 42
56 round(42.0,1) #returns 42.0
57 float("42") #can also convert strings to numbers
58 int("cheese") #but not always
59
```

What is a real world application for this?

F-strings for formatting

f-string statement consists of two parts

1. one is the character f or F
2. the other is a string enclosed in quotes

Inside the string, we can write a Python expression between { and } characters that can refer to variables or literal values

Number formatting

To format numbers in print statements, we can use **f-string** or **format()** function.

```
60 #To format numbers we can use f-strings
61 #you can also use the format() function, but its not as straightforward
62 # Method 1 (using format() function):
63 print("Nearest integer of pi is {:.0f}".format(pi))
64 # Method 2 (using an f-string):
65 print(f"Nearest integer of pi is {pi:.0f}")
--
```

In this course, we'll mainly stick with f-strings, however, if the `format()` function makes more sense to you then use that

Number formatting

```
69     #use .2f to round to 2 decimals
70     print(f"pi rounded to 2 decimal is {pi:.2f}")
71
```

How would you round to 3 decimals?

Add % sign

```
74
75 #use the % sign after the format specifier to format a percentage sign (%)
76 #for example, we can round a float, then apply the % sign
77 num = 89.28456
78 print(f"{num:.1f}%")
79
```

Format numbers with commas

```
80 #To add comma separator in numbers use the format specifier {:,}
81 #use {:,.nf} for n decimal places
82 value = 1234567890.23456789
83 # comma separated with 2 decimal places
84 print(f"{value:,.2f}")
85
```

What is a real world application for all this formatting?

String functions

A function is a block of code that performs a specific task. You can pass data, known as parameters, into a function.

```
--  
86  #some common string functions  
87  my_string = "The quick brown FOX jumps over the lazy DOG"  
88  print(my_string)  
89  print(my_string.upper())  
90  print(my_string.lower())  
91  print(my_string.replace("quick","agile").upper())  
92
```

String functions

A function is a block of code that performs a specific task. You can pass data, known as parameters, into a function.

```
--  
86  #some common string functions  
87  my_string = "The quick brown FOX jumps over the lazy DOG"  
88  print(my_string)  
89  print(my_string.upper())  
90  print(my_string.lower())  
91  print(my_string.replace("quick","agile").upper())  
92  
93  #note the following will produce an error. do you know why?  
94  print(my_string.upper)  
95
```

More string functions

```
96  #more string functions; you can find more via Google
97  a_string = "More functions to consider!"
98  print(a_string)
99  print(a_string.title()) # to capitalize the first letter of every word
100 print(a_string.find("c")) # returns the index of the first occurrence of the letter (or -1 if not found)
101 print(a_string.startswith("M")) # returns true if a string starts with the letter & it is case sensitive
102 print(a_string.endswith("r")) # returns true if a string ends with the letter & it is case sensitive
103 print(a_string.split()) # splits a string into lists of smaller substrings
104 print(a_string.swapcase()) # reverses casing in a string
105
```

Run each of these to see the result

Getting user input

You can ask for user information with the **input()** function. To capture what the user types & can be stored in a variable.

```
106 # getting input from a user
107 name = input("What's your name? ")
108 age = int(input("How old are you? "))
109 age_in_dog_years = age * 7
```

Getting user input

You can ask for user information with the **input()** function. To capture what the user types & can be stored in a variable.

```
106  # getting input from a user
107  name = input("What's your name? ")
108  age = int(input("How old are you? "))
109  age_in_dog_years = age * 7
110
111  print(f"Hi {name}! You're {age_in_dog_years} in dog years. Woof!")
```

Questions about these?

- comments
- variables
- print()
- math
- data types
- type conversion
- string functions
- input()

Challenge 1:

Write Python code that will **ask a user their favorite song** and then **print their response back**, followed by the name of ***your* favorite song**. Your code should include comments.

One possible solution

```
1  #This code asks a user for their favorite song, prints their song & my favorite song
2
3  # Ask the user for their favorite song
4  userfavoritesong = input("What is your favorite song?")
5  userfavoritesong = userfavoritesong.title()
6
7  # Print the user's favorite song
8  print(f"{userfavoritesong} is your favorite song. Nice one!")
9
10 # My favorite song details
11 myfavsong = "Scars To Your Beautiful by Alessia Cara"
12
13 # prints my favorite song
14 print("My favorite song is", myfavsong)
```

Challenge 2: discount calculator

Write Python code that asks a user for the price of an item, then calculate these discount amounts: **10%, 15%, 20% and 25%** and the **final sale price for each discount** amount. Include comments in your code.

One possible solution

```
1  # Discount Calculator Challenge
2
3  # This code asks a user for the price of an item
4  # then calculate these discount amounts: 10%, 15%, 20% and 25%
5  # and prints the final sale price for each discount amount
6
7  # first, convert the user's input into a float
8  amount = float(input("How much is the item? "))
9
10 # then calculate the discount amounts
11 print(f"10% off is ${amount * .10}; the sale price is ${amount-(amount * .10)}")
12 print(f"15% off is ${amount * .15}; the sale price is ${amount-(amount * .15)}")
13 print(f"20% off is ${amount * .20}; the sale price is ${amount-(amount * .20)}")
14 print(f"25% off is ${amount * .25}; the sale price is ${amount-(amount * .25)}")
15
```

Let's discuss

What if the user input was unexpected?

How much is the item?

- *apple*
- *(user hits enter/return key)*
- *\$6.50*

Recap

You created a basic Python script to

1. take input from a user `input()`
2. do some work to it `math, type conversions, string functions`
3. get back an output `print()`

Next week

Selection and decision statements in Python